Aegis PQVM User Manual

1. Introduction

Aegis PQVM is a post-quantum cryptography module for deterministic blockchain runtime integrations. It enables production use of modern post-quantum verification and controlled off-chain key-encapsulation workflows through a compact binary interface.

This manual explains how to install, configure, operate, and validate the module in customer environments.

2. What Aegis PQVM Does

Aegis PQVM provides:

- Deterministic detached-signature verification for ML-DSA and FN-DSA.
- Controlled off-chain ML-KEM decapsulation for trusted environments.
- Runtime adapter interfaces for EVM, Substrate, CosmWasm, Solana, Move, and Bitcoin-style verification flows.
- Key lifecycle metadata management and audit logging.
- Policy-driven randomness beacon generation and verification.
- Security and release evidence workflows for regulated delivery pipelines.

3. Supported Algorithms

Family	Variants
ML-KEM	ML-KEM-512, ML-KEM-768, ML-KEM-1024
ML-DSA	ML-DSA-44, ML-DSA-65, ML-DSA-87
FN-DSA	FN-DSA-512, FN-DSA-1024, FN-DSA-padded-512, FN-DSA-padded-1024

4. Runtime Operation Modes

4.1 Deterministic VM Mode

Use this mode for on-chain style execution.

Supported operations:

- ML-DSA detached signature verification
- FN-DSA detached signature verification

Not supported:

- ML-KEM decapsulation with secret-key payloads

4.2 Trusted Off-chain Mode

Use this mode in trusted services where secret material is permitted.

Supported operation:

- ML-KEM decapsulation

5. System Requirements

5.1 Build Requirements

- Rust stable toolchain
- Cargo
- C build toolchain compatible with `cc` crate
- Standard Unix shell environment for scripts

5.2 Optional Tools (Auto-installed by scripts when needed)

- `cargo-audit`
- `cargo-fuzz`
- nightly Rust toolchain (for fuzzing target runs)
- `cargo-cyclonedx`

6. Installation and Build

6.1 Build with Default Source Root

```
cd aegis-pqvm
cargo build --all-features
```

Default source root for cryptographic C sources is:

- `./pqcore`

6.2 Build with Explicit Trusted Source Root

```
cd aegis-pqvm
AEGIS_PQ_SOURCE_ROOT=/path/to/trusted/source/tree cargo build --all-features
```

7. Quick Start Validation

Run core checks:

```
cd aegis-pqvm
cargo test --all-features
cargo clippy --all-targets --all-features --locked -- -D warnings
```

Run complete production quality gates:

```
cd aegis-pqvm
./scripts/run_quality_gates.sh
```

8. AEG1 Payload Basics

Aegis PQVM uses a compact binary payload called `AEG1`.

Structure:

- 4-byte magic: `AEG1`
- 1-byte operation id
- 1-byte algorithm id
- 1-byte argument count
- each argument is length-prefixed with 4-byte big-endian size

Operational limits:

- max payload: 1,048,576 bytes
- max arguments: 8
- max argument size: 131,072 bytes

9. Common Task: Verify ML-DSA Signature

9.1 Workflow Summary

1. Generate or load `public_key`, `message`, and `signature`.
2. Construct AEG1 call with operation `MldsaVerifyDetached`.
3. Dispatch through runtime adapter.

4. Decode AEG1 response.
5. Interpret result byte (`1 = valid`, `0 = invalid`).

9.2 Rust Example

```
use aegis_pqvm::integrations::abi::{decode_response, encode_call, Alg, Call,
Op};
use aegis_pqvm::integrations::evm;

let call = Call {
    op: Op::MldsaVerifyDetached,
    alg: Alg::Mldsa44,
    args: vec![public_key, message, signature],
};

let payload = encode_call(&call)?;
let raw = evm::evm_precompile_call(&payload)?;
let body = decode_response(&raw)?.map_err(|(_, m)| m)?;
let valid = body == vec![1u8];
```

10. Common Task: Verify FN-DSA Signature

10.1 Workflow Summary

1. Prepare `public_key`, `message`, `signature` for FN-DSA variant.
2. Construct AEG1 call with operation `FndsaVerifyDetached`.
3. Dispatch through adapter.
4. Decode and evaluate response byte.

10.2 Rust Example

```
use aegis_pqvm::integrations::abi::{decode_response, encode_call, Alg, Call,
Op};
use aegis_pqvm::integrations::substrate::SubstrateIntegration;

let call = Call {
    op: Op::FndsaVerifyDetached,
    alg: Alg::Fndsa512,
```

```
    args: vec![public_key, message, signature],
};

let payload = encode_call(&call)?;
let raw = SubstrateIntegration::dispatch_call(&payload)?;
let body = decode_response(&raw)?.map_err(|(_, m)| m)?;
let valid = body == vec![1u8];
```

11. Common Task: ML-KEM Decapsulation in Trusted Off-chain Service

11.1 Workflow Summary

1. Prepare `ciphertext` and `secret_key` in a trusted service process.
2. Construct AEG1 call with operation `MlkemDecapsulate`.
3. Encode with off-chain encoder.
4. Dispatch with off-chain dispatcher.
5. Decode shared secret bytes from response.

11.2 Rust Example

```
use aegis_pqvm::integrations::abi::{decode_response, dispatch_offchain,
encode_call_offchain, Alg, Call, Op};

let call = Call {
    op: Op::MlkemDecapsulate,
    alg: Alg::Mlkem768,
    args: vec![ciphertext, secret_key],
};

let payload = encode_call_offchain(&call)?;
let raw = dispatch_offchain(&payload)?;
let shared_secret = decode_response(&raw)?.map_err(|(_, m)| m)?;
```

12. Runtime Adapter Usage

12.1 EVM

- Entrypoint: `evm::evm_precompile_call(payload)`
- Gas estimator: `evm::evm_gas_cost(payload)`
- Payload format: AEG1

12.2 Substrate

- Entrypoint: `SubstrateIntegration::dispatch_call(call_data)`
- Payload format: AEG1

12.3 CosmWasm

- Entrypoint: `CosmwasmIntegration::call_contract(contract, message)`
- Message must be contract-bound envelope:
  - `CWB1 || contract_len_be_u32 || contract || aeg1_payload`
- Envelope creation helper:
  - `CosmwasmIntegration::encode_bound_message(contract, aeg1_payload)`

12.4 Solana

- Entrypoint: `SolanaIntegration::invoke_instruction(ix)`
- Instruction data payload format: AEG1

12.5 Move

- Entrypoint: `MoveIntegration::invoke_entry_function(module, function, args)`
- Requires exactly one AEG1 payload argument.
- Route and payload operation/algorithm must match.

Supported Move routes:

- `aegis::mldsa44_verify_detached`

- `aegis::mldsa65_verify_detached`
- `aegis::mldsa87_verify_detached`
- `aegis::fndsa512_verify_detached`
- `aegis::fndsa1024_verify_detached`

12.6 Bitcoin-style Verification

- Entrypoint: `BitcoinIntegration::verify_script_payload(payload)`
- Payload format: AEG1

13. Key Lifecycle Operations

Aegis PQVM includes lifecycle tracking for key identifiers.

Capabilities:

- register key metadata
- update key usage timestamp
- schedule rotation
- retire keys with reason
- destroy keys
- export append-only audit log in JSONL format

13.1 Minimal Example

```
use aegis_pqvm::key_lifecycle::{AlgorithmFamily, KeyLifecycleManager};

let mut manager = KeyLifecycleManager::new();
let key_id = manager.register_key(AlgorithmFamily::MLDSA87)?;
manager.touch_key(key_id)?;
manager.schedule_rotation(key_id, future_timestamp)?;
manager.retire_key(key_id, "rotation complete")?;
manager.write_audit_log_jsonl("key_audit.jsonl")?;
```

14. Randomness Beacon Operations

The randomness beacon provides chained outputs with auditable proof fields.

14.1 Default Policy

- policy id: `default`
- minimum entropy sources: `2`
- epoch duration: `300` seconds
- hardware entropy requirement: enabled

14.2 Generate Beacon Output

```
use aegis_pqvm::quantum_randomness_beacon::QuantumBeacon;

let mut beacon = QuantumBeacon::new();
let output = beacon.generate_beacon("default")?;
```

14.3 Verify Beacon Output

```
let verdict = beacon.verify_beacon(&output);
```

14.4 Standalone Verification

```
use aegis_pqvm::quantum_randomness_beacon::verify_beacon_standalone;

let verification_key = beacon.get_verification_key().unwrap();
let verdict = verify_beacon_standalone(&output, verification_key, None);
```

15. Security and Compliance Operations

Run policy and security checks:

```
cd aegis-pqvm
./scripts/check_naming_policy.sh
./scripts/check_no_absolute_paths.sh
./scripts/check_no_ignored_kat.sh
./scripts/check_effective_pqc_source_markers.sh
./scripts/check_traceability_matrix.sh
./scripts/check_independent_security_signoff.sh
```

Run deterministic KAT replay:

```
cd aegis-pqvm
./scripts/run_deterministic_kat.sh
```

Run side-channel boundary review:

```
cd aegis-pqvm
./scripts/run_side_channel_review.sh
```

Run fuzz and fault campaign:

```
cd aegis-pqvm
./scripts/run_fuzz_fault_campaign.sh
```

16. Benchmarking

Generate benchmark report artifacts:

```
cd aegis-pqvm
./scripts/run_benchmark_report.sh
```

Outputs:

- `docs/benchmark/pqvm_benchmarks.csv`
- `docs/benchmark/pqvm_benchmarks.json`
- `docs/benchmark/PQVM_COMPARATIVE_BENCHMARK_REPORT.md`
- timestamped raw logs in `docs/benchmark/`

17. Release Artifact Generation

Generate checksums and SBOM:

```
cd aegis-pqvm
./scripts/generate_release_manifest.sh
./scripts/generate_sbom.sh
```

Create customer bundle:

```
cd aegis-pqvm
./scripts/package_customer_bundle.sh
```

Primary output locations:

- checksums: `artifacts/checksums/`
- SBOM: `artifacts/sbom/`
- package: `artifacts/package/`

18. Troubleshooting

18.1 `payload must not be empty`

Cause:

- Adapter entrypoint received empty bytes.

Resolution:

- Ensure message serialization includes full AEG1 request.

18.2 `bad magic` or `payload too small`

Cause:

- Input is not valid AEG1.

Resolution:

- Rebuild payload using `encode_call()` or `encode_call_offchain()`.

18.3 Deterministic adapter rejects ML-KEM decapsulation

Cause:

- Secret-key operation attempted in deterministic interface.

Resolution:

- Move decapsulation to trusted off-chain service using `dispatch_offchain()`.

18.4 CosmWasm contract mismatch error

Cause:

- Envelope contract bytes differ from runtime contract argument.

Resolution:

- Regenerate envelope with exact contract identifier used at dispatch.

18.5 Move route mismatch error

Cause:

- Move module/function does not match payload operation or algorithm.

Resolution:

- Align payload operation/algorithm with supported route map.

18.6 Quality gate failure

Cause:

- At least one required gate failed.

Resolution:

1. Re-run the failing command directly.
2. Fix underlying issue.
3. Re-run `./scripts/run_quality_gates.sh`.

19. Command Reference

Command	Purpose
`cargo build --all-features`	Build module with all supported capabilities
`cargo test --all-features`	Execute test suites
`./scripts/run_quality_gates.sh`	Execute full release gate pipeline
`./scripts/run_deterministic_kat.sh`	Run deterministic known-answer-test replay
`./scripts/run_side_channel_review.sh`	Run side-channel boundary checks
`./scripts/run_fuzz_fault_campaign.sh`	Run fuzzing and deterministic fault campaign
`./scripts/run_benchmark_report.sh`	Generate benchmark artifacts
`./scripts/generate_release_manifest.sh`	Generate checksum manifest
`./scripts/generate_sbom.sh`	Generate SBOM
`./scripts/package_customer_bundle.sh`	Build distributable customer bundle

20. Documentation Map

- Developer guide: `docs/developer/DEVELOPER_GUIDE.md`

- Technical specification: `docs/specification/TECHNICAL_SPECIFICATION.md`
- Threat model: `docs/security/THREAT_MODEL.md`
- Traceability matrix: `docs/security/TRACEABILITY_MATRIX.md`
- Release policy: `docs/release/RELEASE_POLICY.md`